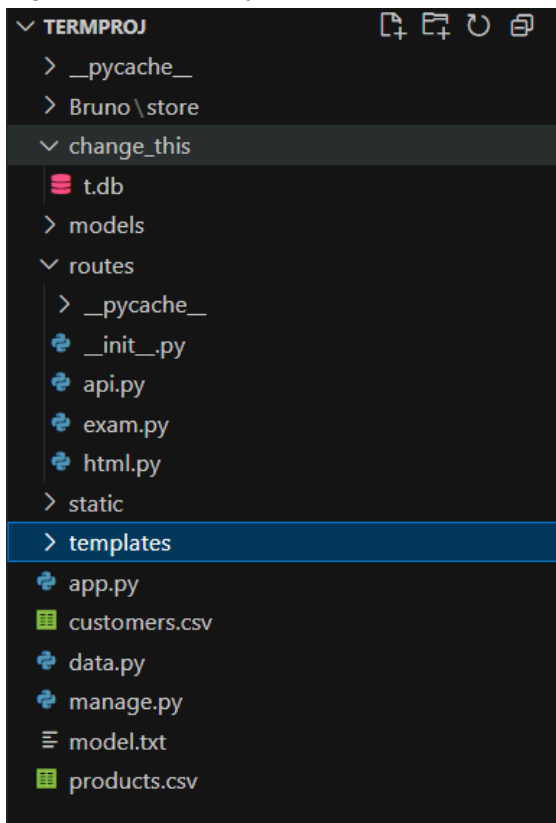


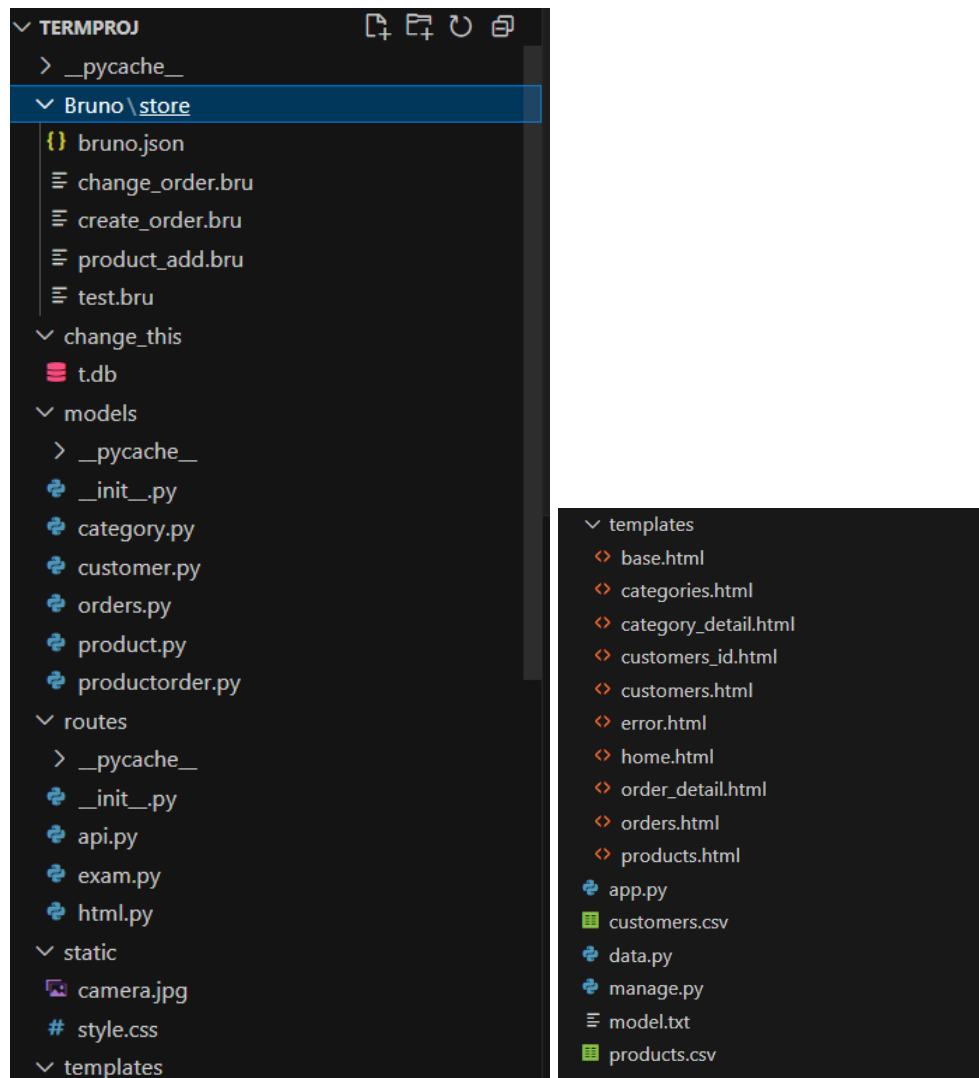
Dropping all tables

```
PS C:\Users\jians\Downloads\School-Home\python\TermProj> python
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from manage import drop
>>> drop
<function drop at 0x000002178A0180E0>
>>> app.app_context().push()
Traceback (most recent call last):
  File "<python-input-2>", line 1, in <module>
    app.app_context().push()
    ^^^
NameError: name 'app' is not defined
>>> from app import app
>>> app.app_context().push()
>>> drop()
>>> []
```

I added an “exam.py” into my routes folder to house the routes with /exam prefixes. Exam.py is exported inside of `init.py` and a simple “/” route is created. Made sure that exam.py blueprint is registered in app.py



Screenshots of full directory



Lines highlighted/colored yellow are the changes for Customer

```
from data import db
from .orders import Order

class Customer(db.Model):
    __tablename__ = "customers"

    id=db.mapped_column(db.Integer, primary_key=True)
    name=db.mapped_column(db.String)
    phone=db.mapped_column(db.String)
    money=db.mapped_column(db.DECIMAL(6,2), default=0)
    status=db.mapped_column(db.String, default="regular")

    orders = db.relationship("Order", back_populates="customers")

    def Completed(self):
        stmt = db.select(Order).where(Order.customers.has(Customer.id == self.id)).order_by(Order.completed.desc())
        result = db.session.execute(stmt).scalars()
        return [x.to_json() for x in result if x.completed]

    def pending(self):
        stmt = db.select(Order).where(Order.customers.has(Customer.id == self.id)).order_by(Order.created.asc())
        result = db.session.execute(stmt).scalars()
        return [x.to_json() for x in result if not x.completed]

    def to_json(self):
        return {
            "id": self.id,
            "name": self.name,
            "phone": self.phone,
            "Money": self.money,
            "status": self.status,
            "pending_orders": self.pending(),
            "completed_orders": self.Completed()
        }
```

Lines highlighted/colored yellow are changes for Order

```
from data import db

class Order(db.Model):
    __tablename__ = "orders"

    id = db.mapped_column(db.Integer, primary_key=True)
    created = db.mapped_column(db.DateTime, nullable=False, default=db.func.now())
    completed = db.mapped_column(db.DateTime, nullable=True, default=None)
    amount = db.mapped_column(db.DECIMAL(6, 2), nullable=True, default=None)
    option = db.mapped_column(db.String, default="pickup")

    items = db.relationship("ProductOrder", back_populates='orders')
    customer_id = db.mapped_column(db.Integer, db.ForeignKey("customers.id"))
    customers = db.relationship("Customer", back_populates="orders")

    def estimate(self):
```

```
    def to_json(self):
        return {
            "id": self.id,
            "completed": bool(self.completed),
            "completed_date": self.completed,
            "created": self.created,
            "name": self.customers.name,
            "estimated_total": self.estimate(),
            "deliver": bool(self.option),
            "products": [x.to_json() for x in self.items]
        }
```

Customer table (empty)

id	name	phone	money	status
Filter	Filter	Filter	Filter	Filter

part3.py

```
part3.py > make
from models import Customer
from data import db
from app import app

def make():
    names = [["Jian Somera", "604-720-3851"], ["Tim Guicherd", "666-888-9999"]]
    for i in range(len(names)):
        result = db.session.execute(db.select(Customer).where(Customer.name == names[i][0])).scalar()
        if not result:
            customer = Customer(name=names[i][0], phone=names[i][1])
            db.session.add(customer)
    db.session.commit()

if __name__ == "__main__":
    app.app_context().push()
    make()
```

part3.py resulting tables

Rows: 2		Filter 2 rows			
	id	name	phone	money	status
	1	Jian Somera	604-720-3851	0	regular
	2	Tim Guicherd	666-888-9999	0	regular
+	3				

Giving self customer premium and more money

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/exam/customers/1`. The request body is a JSON object with `"money": 1000000` and `"premium": true`. The response is a 200 OK status with a JSON body containing customer details: `"Money": "1000000.00"`, `"completed_orders": []`, `"id": 1`, `"name": "Jian Somera"`, `"pending_orders": []`, `"phone": "604-720-3851"`, and `"status": "premium"`.

```
PUT http://localhost:3000/exam/customers/1
```

Params Body * Headers Auth * Vars Script

Assert Tests Docs JSON Prettify

```
1 {
2   "money": 1000000,
3   "premium": true
4 }
```

Response Headers 5 Timeline Tests 200 OK 102ms 164B

```
1 {
2   "Money": "1000000.00",
3   "completed_orders": [],
4   "id": 1,
5   "name": "Jian Somera",
6   "pending_orders": [],
7   "phone": "604-720-3851",
8   "status": "premium"
9 }
```

Products and Category table

Rows: 5						Filter
	id	name	price	inventory	category	
1	1	one dollar	1	0	1	
2	2	three dollars	3	0	1	
3	3	five dollars	5	0	1	
4	4	expensive	500	0	2	
5	5	passing grade	1000	0	2	
+	6					

Rows: 2			
	id	name	
1	1	price	
2	2	final	
+	3		

New orders Jian

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/api/orders`. The request body is a JSON object with the following structure:

```
1 {
2   "customer_phone": "604-720-3851",
3   "items": [
4     ["one dollar", 2],
5     ["expensive", 1]
6   ]
7 }
8
9 // {
10 //   "customer_phone": "666-888-999",
11 //   "items": [
12 //     ["five dollars", 1],
13 //     ["passing grade", 1]
14 //   ]
15 // }
16
```

The response is a JSON object with the following structure:

```
1 {
2   "completed": false,
3   "completed_date": null,
4   "created": "Mon, 14 Apr 2025 14:53:30 GMT",
5   "deliver": true,
6   "estimated_total": "502.00",
7   "id": 1,
8   "name": "Jian Somera",
9   "products": [
10    {
11      "inventory": 0,
12      "name": "one dollar",
13      "price": "1.00",
14      "quantity": 2
15    },
16    {
17      "inventory": 0,
18      "name": "expensive",
19      "price": "500.00",
20      "quantity": 1
21    }
22  ]
23 }
```

New order Tim

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/api/orders`. The request body is a JSON object with the following structure:

```
1
2 {
3   "customer_phone": "666-888-9999",
4   "items": [
5     ["five dollars", 1],
6     ["passing grade", 1]
7   ]
8 }
9
```

The response is a JSON object with the following structure:

```
1 {
2   "completed": false,
3   "completed_date": null,
4   "created": "Mon, 14 Apr 2025 14:55:20 GMT",
5   "deliver": true,
6   "estimated_total": "1005.00",
7   "id": 2,
8   "name": "Tim Guicherd",
9   "products": [
10    {
11      "inventory": 0,
12      "name": "five dollars",
13      "price": "5.00",
14      "quantity": 1
15    },
16    {
17      "inventory": 0,
18      "name": "passing grade",
19      "price": "1000.00",
20      "quantity": 1
21    }
22  ]
23 }
```